

Computer Networks

Part 4

Transport Layer

Summary

- Overview
- Transport Control Protocol (TCP)
- User Datagram Protocol (UDP)
- Port Numbers and Sockets

Transport Layer

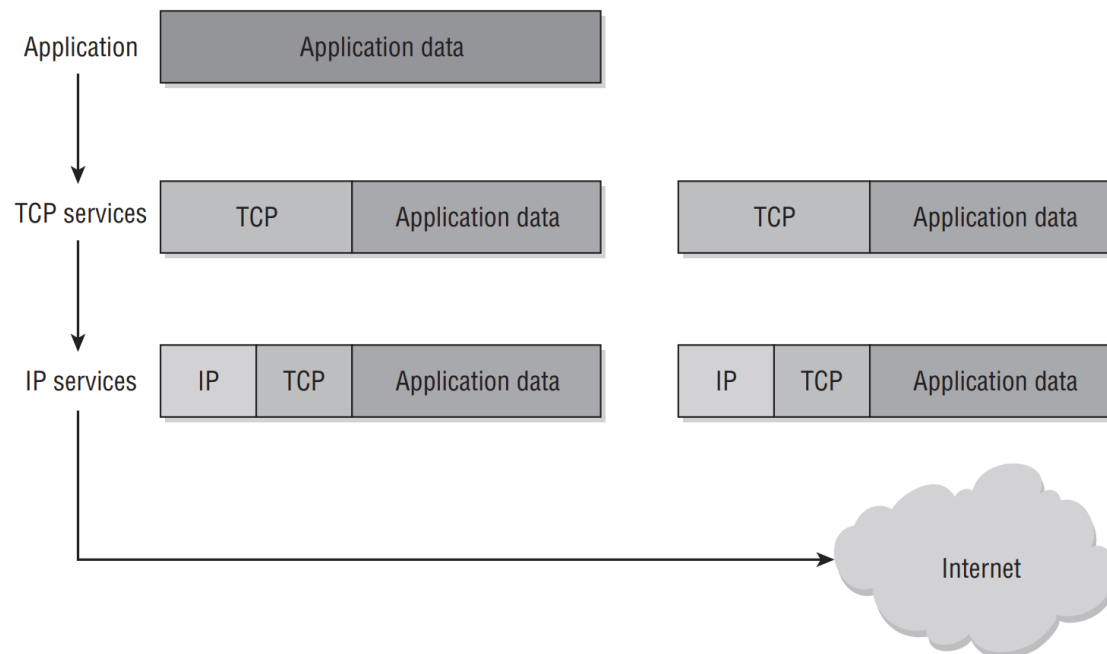
Understanding the Transport Layer

- The transport layer is designed to provide end-to-end control and information transfer of data sent from upper-layer applications.
- The goal of the transport layer is to relieve the upper layers from the need to provide reliable data transfer over an unreliable network protocol (such as Internet Protocol (IP)).
- In the TCP/IP stack, the Transmission Control Protocol (TCP) and the User Datagram Protocol (UDP) provide services very similar to the OSI transport protocols and are therefore often referred to as transport or Layer 4 protocols.
- If the application needs a high level of service, such as reliable data transfer and flow control, the application typically uses TCP for data transfer as it provides these capabilities.
- If an application needs a simpler service with less overhead, the application may use UDP and perform any additional checking of data delivery itself.
- Although most higher-level protocols use TCP or UDP, there are a few higher-level protocols that do not use either of them. For example, the Open Shortest Path First (OSPF) routing protocol uses IP datagrams directly and does not use a transport layer protocol at all.

Transport Layer

Understanding the Transport Layer

- Supposing an application that uses TCP to transport data:
 - The application data is first passed to the TCP services layer.
 - The TCP layer then divides the application data into segments, if necessary (if, e.g., the amount of application data is larger than a single TCP segment can accommodate). Each TCP segment contains a TCP header.
 - The size of the segments is based on the maximum transmission unit (MTU) size of the Layer 2 networks that are expected to be used for the transfer.
 - The TCP segments are then passed to the IP services layer, where they are delivered across the network as IP datagrams.



Transport Layer

Transport Control Protocol (TCP)

- TCP is by far the most important transport layer protocol and is second in importance of topics only to IP.
- TCP provides a wealth of features that allow for reliable communication across virtually any infrastructure.
- The primary purpose of TCP is to provide reliable communications between application services.
- Because the lower layers are unreliable, TCP must guarantee the delivery of the data
 - Note that saying that a lower layer is unreliable doesn't mean that the lower layers often fail.
 - In fact, networks today are often very reliable.
 - It simply means that the lower layers do not guarantee that a packet will be delivered.

Transport Layer

Transport Control Protocol (TCP)

- The original TCP specification is provided in RFC 793.
- This RFC specifies the key functionality provided by TCP.
- Some of these functions are:
 - **Data Transfer:** From the application-services viewpoint, TCP provides a contiguous stream of data through the network.
 - **Reliability:** TCP uses sequence numbers, which count each byte transmitted, and TCP waits for an acknowledgment from the receiving host. If the acknowledgment is not received within a specific interval, the data is re-transmitted.
 - **Flow Control and Congestion Control:** Flow control refers to the capability of the receiver to control the rate at which data is sent by the sender by setting a “window size.” Congestion control refers to the sender’s ability to alter the rate at which it is sending based on network conditions such as packet drops.
 - **Multiplexing:** Port numbers are used for multiplexing and de-multiplexing. This means that you can have many applications running simultaneously that all use TCP, with each having its own port number.
 - **Logical Connections:** To support reliability and flow control, TCP must initialize and maintain status information for each connection. The status information contains sockets numbers, sequence numbers, and window size. These components combine to form a logical connection.
 - **Full-Duplex:** A TCP connection is full-duplex: either end may transmit data at any time.

Transport Layer

The TCP Header

- **Source and Destination Ports:** Port addresses identify the upper-layer applications that use the connection.
- **Sequence Number:** each byte of data is assigned a sequence number. This 32-bit number ensures that data is correctly sequenced.
- **Acknowledgment Number:** This 32-bit number indicates the next sequence number that the sending device expects from the other station. Note that this is the next sequence number it is expecting, not the current sequence number. This is a key point to understand.
- **HLEN:** The header length provides the number of 32-bit words in the header.
- **Reserved:** The value is always set to 0.

Source Port					Destination Port				
Sequence Number									
Acknowledgement Number									
HLEN	Res.	URG	ACK	PSH	RST	SYN	FIN	Window	
Checksum					Urgent Pointer				
Options and Padding									
Data									

Transport Layer

The TCP Header

Code Bits - The following flags indicate the type of header:

- **URG** - Indicates that there is data in the segment that should be given priority and passed immediately to the application (e.g., [Ctrl]+C to abort an operation).
 - **ACK** - Acknowledgment, always present except in the initial packet of a TCP connection.
 - **PSH** - Used by the application to indicate to TCP that the data should be transmitted immediately rather than buffered (e.g., to finish filling a segment). Also indicates to the receiver that data should be given immediately to the application. Intended to reduce overall latency that could be introduced by TCP.
 - **RST** - Re-sets the connection. This essentially tears down the connection.
 - **SYN** - Synchronizes sequence numbers, only used during connection establishment.
 - **FIN** - End of data. Closes down one side of the connection.
-
- **Window**: The window indicates the number of octets that the sender of the ACK is willing to accept before an acknowledgment is sent.
 - **Checksum**: The checksum is used to verify the integrity of the TCP segment. The checksum calculation is performed on the TCP pseudo-header and data. The pseudo-header is the IP source and destination addresses, TCP header, and the TCP data.
 - **Urgent Pointer**: The urgent pointer indicates the location of the urgent data. When the URg bit is set, this data is given priority over other data in the data stream. It should be noted that this and the URg flag are rarely used in modern applications.
 - **Options**: Several options are defined for TCP. The most common is the TCP Maximum Segment Size (MSS)

Transport Layer

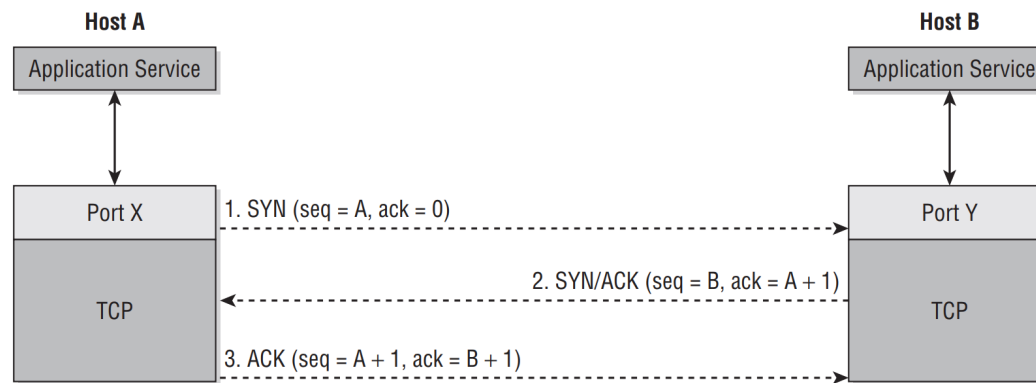
The TCP Header

- The **MSS** defines the largest segment that will be sent on the TCP connection.
- The value is an estimate by the TCP of the size of datagrams that can be accommodated on the connection without fragmentation.
- Usually, each side sends the MTU value of its Layer 2 connection in the MSS field. The lower of the two values is then used by both sides as the MSS. For example, the MTU of ethernet is 1,514, while the MTU of a PPP connection might be 500. If both ends sent their respective MTUs, then 500 would be chosen as the MSS since the lowest MSS is always chosen.
- However, the problem with determining the MSS using only the two endpoints is that there may be a link in the middle of the connection that has a smaller MTU than either end. In this case, all full-size packets will have to be fragmented by IP to traverse this link.

Transport Layer

Establishing a TCP Connection—The Three-Way Handshake

- Before TCP can transmit any data, it must first establish a connection from the sender to the receiver. This process is called the **three-way handshake**.



- Host A sends a SYN request with a starting sequence number, an acknowledgement number set to zero, and the SYN bit set.
- Host B acknowledges this by sending a response with its own sequence number, an acknowledgement number equal to Host A's sequence number plus 1, and the SYN and ACK bits set.
- Host A then responds with a new sequence number equal to its initial sequence number plus 1, an acknowledgement number equal to Host B's sequence number plus 1, and the ACK bit set.
- Once these three packets are exchanged, the handshake is complete and communication can begin.

Transport Layer

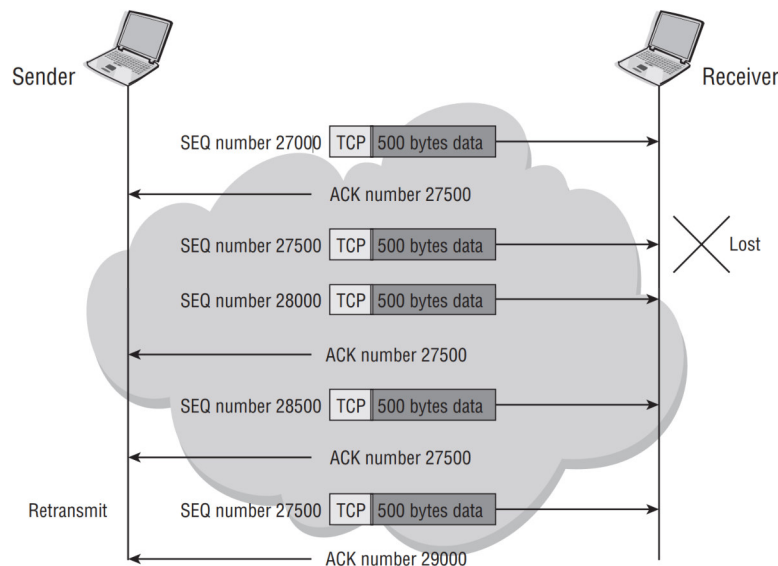
TCP Reliable Data Transfer

- TCP will order the data for the application, even if the data arrives out of order.
- Since TCP ensures delivery even if some packets are dropped, it will wait to receive missing data before passing it to the upper-layer application.
 - This technique is known as ***positive acknowledgement with re-transmission*** because TCP actively acknowledges only the data that was actually received; it does not request information that has not been received.
 - It is up to the sender to notice out-of-order or duplicate acknowledgements and re-send missing data.
- Each segment that is sent by TCP has an identifying sequence number transmitted in the TCP header. This sequence number indicates the number of the first byte of data in the overall data stream for this connection.
- The receiver acknowledges receipt of this data by transmitting an acknowledgement number that indicates the next byte of data in the stream that the receiver expects to receive.
- If some of the data is lost, the receiver will continue to send the same acknowledgement number that indicates the bytes that were received successfully.
- The sender maintains a re-transmission timer. If the sender does not receive an acknowledgement for some bytes of data that were sent, the data will be re-transmitted when the re-transmission timer expires.

Transport Layer

TCP Reliable Data Transfer

- Because the TCP segments are transmitted over an unreliable network service (IP network), the segments may arrive at the destination in a different order from that in which they were originally sent.
- The sequence numbers are used by the receiver to reconstruct the data stream and ensure that the data is provided to the application in the same order that the data was sent.



- TCP sends SEQ number 27000, which is acknowledged by the receiver, and the receiver indicates it expects to receive 27500 next.
- The sender then sends two segments with numbers 27500 and 28000, but 27500 is lost.
- The receiver does not acknowledge that it received 28000 because this would indicate to the sender that it received both 27500 and 28000.
- Instead, the receiver repeats the acknowledgement, indicating that it is still expecting 27500, which is how the sender recognizes that 27500 was lost and that it needs to re-send.
- Once 27500 is received, the receiver requests segment 29000 since it has already received 28000 and 28500 .

Transport Layer

TCP Flow Control

- If a TCP sender waits to receive acknowledgement for each segment that it sends before sending another segment, the effective throughput of the connection can be greatly limited over the bandwidth that is supported by the transmission media.
- This is not significant on a high-speed LAN because the acknowledgements are received very quickly. However, if the network Round Trip Time (RTT) is long, the sender may spend a significant amount of time waiting for acknowledgements.
- To increase the overall throughput on TCP connections, TCP allows the sender to send more than one segment without waiting for an acknowledgement. This provides a higher overall throughput because TCP spends less time waiting for acknowledgements.
- However, there is a danger of overwhelming the TCP receiver with too much data. To avoid overwhelming the receiver, the amount of data that can be sent to the receiver before receiving an acknowledgment must be controlled.
- To accomplish this, the received data is buffered in a preset amount of buffer space until it is requested by the application.

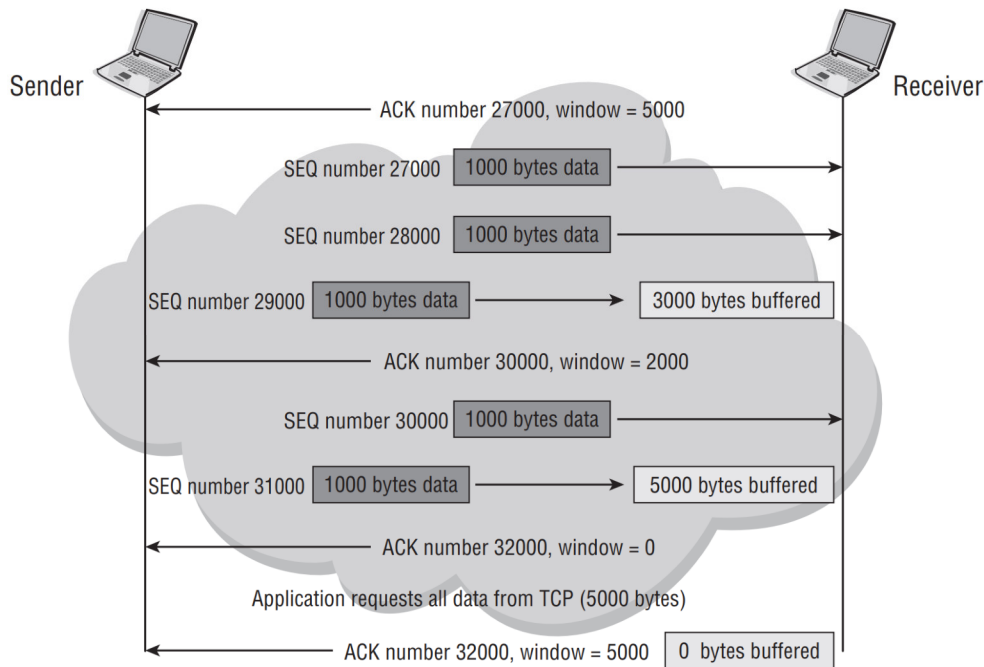
Transport Layer

TCP Flow Control

- The amount of buffer space is specified in the TCP header **window-size** parameter. When the receiver sends an acknowledgement, the receiver's TCP header sets the value of the **window-size** parameter to specify the amount of buffer space (in bytes) that is available.
- This is the maximum amount of data that the sender can send before it receives the next acknowledgement.
- If the receiver's buffer becomes full, the receiver sends a window size of 0 and the sender cannot transmit any more data.
- When the receiving application requests the data and the TCP buffer space is available, the receiver sends an updated window size and the sender can start to transmit more data.
- The window value is always set by the receiver, which provides a flow control mechanism for the receiver.

Transport Layer

TCP Flow Control



- The sender is transmitting data faster than the host can receive it.
- The first advertised window size is 5,000 in the first acknowledgment. After additional data is received, the receiver reduces its window size to 2,000 in the next acknowledgment.
- After still more data is received, the receiver reduces its window size to 0, effectively halting the sender from sending any additional data until the receiver's buffer is cleared and it sends a non-zero window size to the sender.
- At this point, the sender can resume its transmission.

Transport Layer

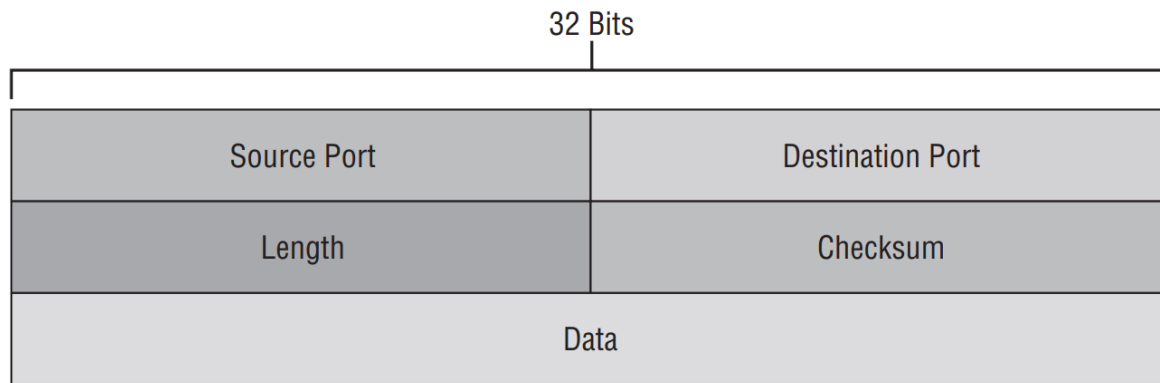
User Datagram Protocol (UDP)

- UDP provides a simple, connectionless, unreliable datagram delivery service. The service is similar to the service that is provided by IP, although UDP has port addresses to support multiplexing between different applications.
- UDP is used when an application does not need a reliable transfer mechanism or if the application needs to avoid the additional overhead of TCP.
- The term unreliable refers simply to the fact that UDP does not provide flow control, acknowledgement, or re-transmission capabilities like those provided by TCP. These capabilities require additional overhead and slow down communication.
- Therefore, UDP may be used for applications in which real-time factor is more critical than packet loss, such as Voice over IP or DnS.

Transport Layer

The UDP Header

- The UDP header contains only port, length, and checksum fields. There are no synchronization, sequence, or acknowledgment fields.
- It does not have many of the fields that the TCP header has because UDP is a much simpler transport protocol.
- The UDP header is obviously very simple compared to the TCP header. There are no synchronization, sequence, or acknowledgment fields.
- UDP is similar to TCP in its use of port numbers to identify the receiving and sending application processes. UDP uses the port numbers in the multiplexing and de-multiplexing operations.
 - This function is necessary to allow UDP (and TCP) to provide services to many upper-layer applications at the same time.



Transport Layer

Port Numbers and Sockets

- Both TCP and UDP use port numbers to multiplex and de-multiplex multiple applications.
- Some of the more common applications for both transport protocols are listed in the figure:

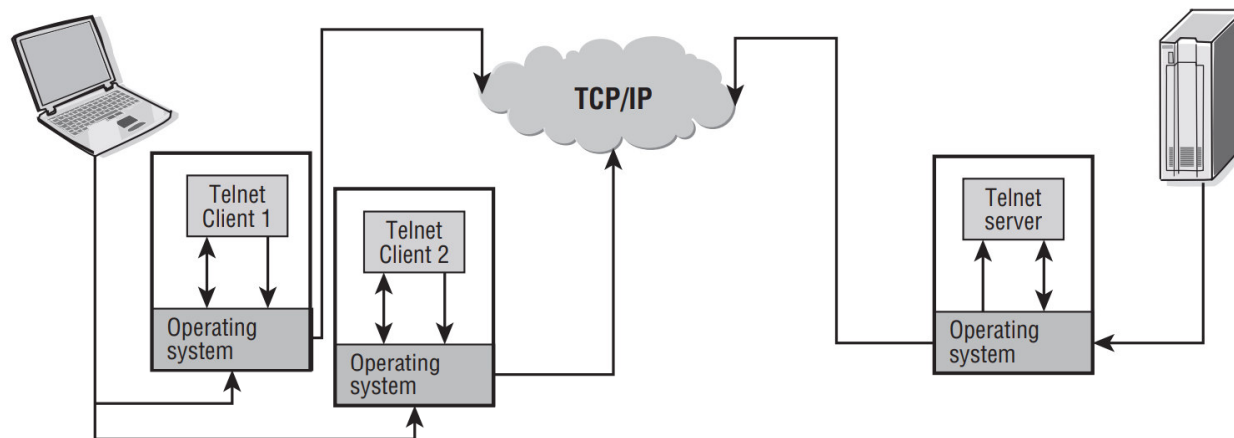
Application Services									
FTP	Telnet	HTTP	SMTP	IMAP	DOOM	DNS		TFTP	Gopher
21	23	80	25	143	666	53	53	69	70
TCP							UDP		

- Ports identify an application service. This allows the transport layer to differentiate between application services.
- There are two types of port numbers:
 - **Well-Known Ports:** Well-known port numbers belong to standard services. The port numbers range from 1 to 1023 and are assigned by the IANA.
 - **Ephemeral Ports:** Client applications do not require well-known port numbers because they initiate communications with servers to well-known ports; they do not usually need to listen for connections. The host system allocates each client process a port number for only as long as the process needs the port number and then may reuse it later (hence the term ephemeral). The port numbers range from 1024 to 65535 and are not controlled by the IANA. Because the host dynamically assigns the port number to the client application, the port number may vary each time that the client application is started.

Transport Layer

Port Numbers and Sockets

- Sockets are very similar to ports, but a socket is actually a combination of IP address and port information.
- Sockets are used to identify the network connection between applications.
- The combination of a protocol (i.e., TCP or UDP), IP address, and port number uniquely identifies a socket on each end.



Enable Telnet client 1 application.
Create client socket TCP,138.120.191.233,15633.
Connect to server.

Enable Telnet client 2 application.
Create client socket TCP,138.120.191.233,15322.
Connect to server.

Enable Telnet server application.
Create socket address TCP,138.120.168.100,23.
Listen to client requests: incoming request from Client 1
Conversation: TCP, 138.120.168.100,23,138.120.191.233,15633,
Incoming request from Client 2
Conversation: TCP, 138.120.168.100,23,138.120.191.233,15322.

Transport Layer

Other Transport Protocols and Their Release Years

Protocol	Release Year	Milestone / Standard
RTP (Real-time Transport Protocol)	1996	RFC 1889 (obsoleted by RFC 3550 in 2003)
SCTP (Stream Control Transmission Protocol)	2000	RFC 2960 (updated by RFC 4960 in 2007)
DCCP (Datagram Congestion Control Protocol)	2006	RFC 4340
UDT (UDP-based Data Transfer Protocol)	2005	Developed by University of Illinois; academic protocol
FASP (Fast And Secure Protocol, IBM Aspera)	2006	Proprietary; introduced by Aspera Inc.
MPTCP (Multipath TCP)	2013 (Linux support), 2020 (RFC)	RFC 8684
RUDP (Reliable UDP – Microsoft, etc.)	2000s	Multiple versions, non-standard
QUIC	2013 (Google draft), 2021 (RFC)	RFC 9000

Transport Layer

Main features of MPTCP protocol

Feature	Description
Multipath Capability	Allows a single TCP connection to use multiple paths simultaneously (e.g., Wi-Fi + LTE).
Backward Compatibility	Compatible with standard TCP; falls back to TCP if the other endpoint doesn't support MPTCP.
Subflows	Splits the connection into multiple subflows, each behaving like a normal TCP connection.
Improved Throughput	Aggregates bandwidth from multiple interfaces for higher data rates.
Resilience and Failover	Maintains connection if one path fails, enhancing reliability.
No Application Changes Needed	Works transparently with existing applications via OS-level support.
Congestion Control	Employs coupled congestion control to balance traffic across paths fairly.
Use Cases	Mobile devices, data center redundancy, seamless handover between networks.
Security	Uses TCP-level authentication (e.g., HMACs) between subflows to prevent session hijacking.
Standardization	Defined in [RFC 6824 (now obsolete)] and updated in [RFC 8684].

Transport Layer

Main features of QUIC protocol

Feature	Description
Protocol Base	Runs on UDP instead of TCP
Handshake Latency	Supports 0-RTT and 1-RTT handshakes for faster connection setup
Encryption	TLS 1.3 integrated directly into transport layer
Multiplexing	Multiple streams with no Head-of-Line (HoL) blocking
Connection ID	Uses stable Connection ID to allow connection migration across networks
User-Space Implementation	Typically implemented in user space , making upgrades easier
Packet Loss Handling	Custom congestion control and selective retransmission per stream
Mobility Support	Seamless IP address changes without reconnecting
Security	Encrypts almost all control information; no fallback to insecure protocols
Application Protocol	Primary transport for HTTP/3

Transport Layer

Local Port Forwarding

- **Local port forwarding** is an SSH feature that allows you to forward a port **on your local machine** to a port **on a remote server** through a secure SSH connection
- It is often used to access services running on a remote network that are not directly accessible due to firewalls or private network configurations
- The SSH server acts as a middleman to tunnel your local traffic to the destination

Syntax (SSH Command):

```
ssh -L [local_port]:[destination_host]:[destination_port] user@ssh_server
```

Example:

```
ssh -L 8080:web-app.corp.local:80 user@gateway.company.com
```

This forwards local port 8080 to port 80 on the remote server. When you visit `http://web-app.corp.local:8080` in your browser:

- SSH client captures this connection
- forwards it through SSH tunnel to `gateway.company.com`
- remote SSH daemon forwards to `web-app.corp.local:80` (the web server)
- web content returns through the same encrypted path

- The **local port forwarding** mechanism provides several security advantages:
 - All forwarded traffic is encrypted within the SSH tunnel
 - Remote services appear as local services, bypassing firewalls
 - Authentication occurs at the SSH level
 - Traffic analysis becomes much more difficult for observers

Transport Layer

Remote Port Forwarding

- **Remote port forwarding** works in the opposite direction of local port forwarding - it creates a tunnel from a port on the remote SSH server back to your local machine or another host accessible from your local network.
- Unlike **local port forwarding** (which forwards traffic from the local machine to a remote server), **remote port forwarding** sends traffic in the opposite direction - from a remote server to your local machine.

Syntax (SSH Command):

```
ssh -R [remote_port]:[destination_host]:[destination_port] user@ssh_server
```

Example:

```
ssh -R 8080:localhost:3000 user@public-server.com
```

This makes your local service on port 3000 accessible from the internet via port 8080 on the remote server:

- external users connect to public-server.com:8080
- SSH server captures these connections
- forwards them through the tunnel to your machine
- your SSH client forwards to localhost:3000 (your local web app)
- your app's responses travel back through the same encrypted path

➤ Common use cases:

- Share a local web server with remote colleagues
- Demo local applications without deploying
- Access home servers from behind NAT routers
- Connect to services behind corporate firewalls

Transport Layer

Port Forwarding on a Wireless Router

➤ NAT and Private IPs

- Home routers use NAT (Network Address Translation) to allow multiple devices to share a single public IP
- Devices connected to the router (e.g., laptops, servers, IP cameras) are assigned private IP addresses (e.g., 192.168.1.10).
- NAT blocks incoming traffic by default, unless it's in response to an outgoing connection.

➤ The Problem

- Suppose you're hosting a web server on 192.168.1.100:80 and want someone on the Internet to access it.
- Because of NAT, the router doesn't know which internal device to forward unsolicited incoming traffic to.

➤ The Solution: Port Forwarding

- You configure the router to forward incoming packets from a specific port on the router's public IP to a specific device (local IP) and port on your LAN.

➤ Example Scenario

- You want to allow Internet users to access a game server running on your PC at 192.168.1.100:25565 (Minecraft, for example):
- Your public IP (from ISP): 82.123.45.67
- Your local game server: 192.168.1.100:25565

Router config:

External Port: 25565

Internal IP: 192.168.1.100

Internal Port: 25565

Protocol: TCP/UDP or both

Now, any packet sent to 82.123.45.67:25565 will be forwarded to 192.168.1.100:25565.